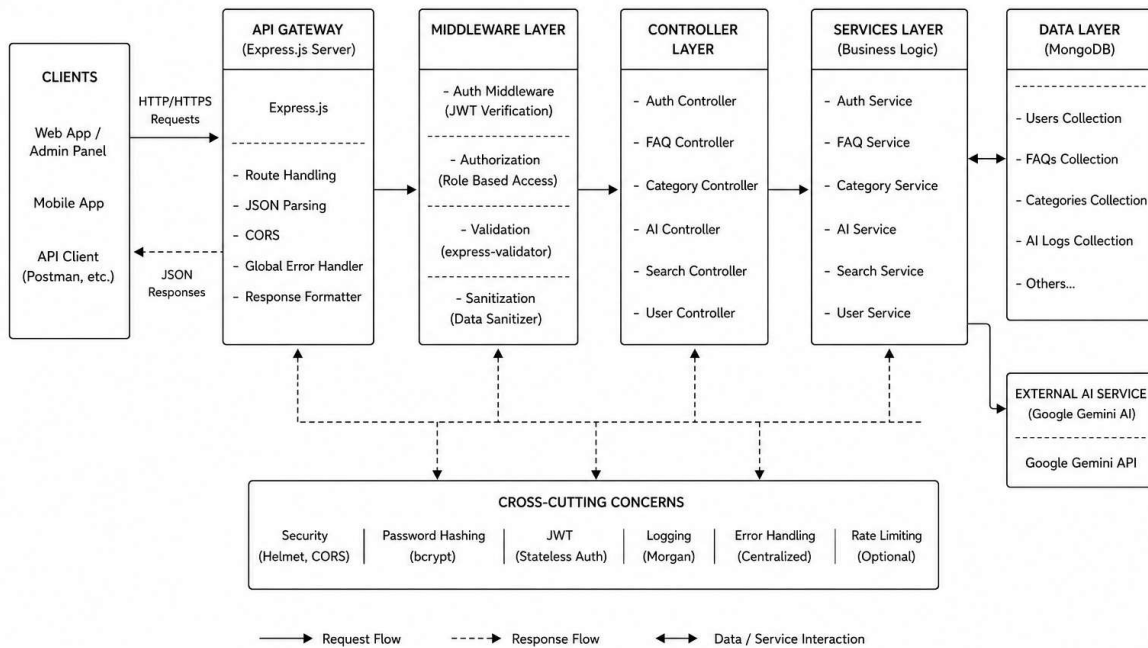


AI FAQ ASSISTANT

Image-Based Technical Project Report

A newly written project document created from the supplied architecture, database, code, workflow and API-testing images.

AI FAQ Assistant API – Technical Architecture



Technical architecture shown in the supplied project material.

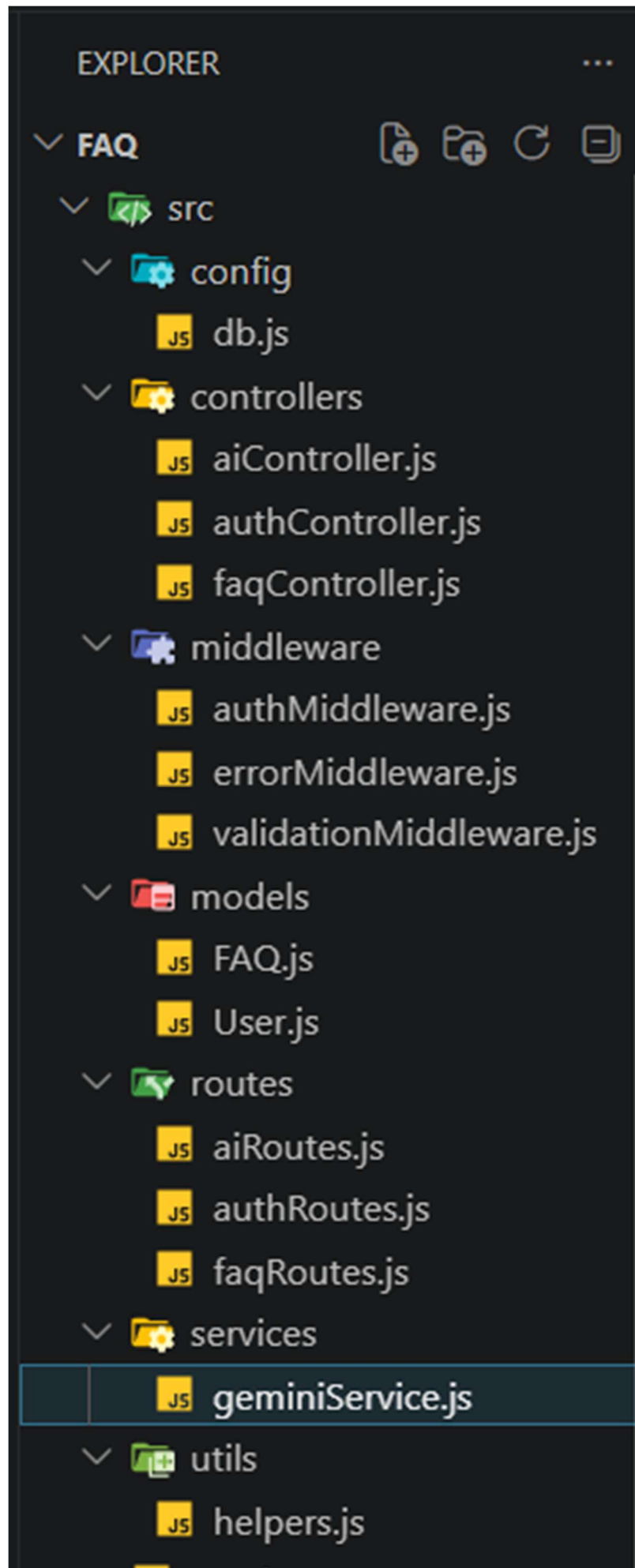
1. Project Overview

The AI FAQ Assistant is presented as a backend service that combines an Express-based HTTP layer, MongoDB persistence, protected routes, and an AI generation service. The supplied visuals show a modular codebase, a controller-oriented request flow, structured data entities, and API checks performed through a REST client.

The project can be understood as a support-content platform: users authenticate, work with FAQ records, search or manage content through API endpoints, and can request AI-assisted FAQ generation. The architecture keeps routing, business logic, persistence, authentication, and AI integration in separate responsibilities.

What the visuals demonstrate

- A layered backend organized around routes, controllers, models, middleware, configuration, and services.
- JWT-style request protection applied to private operations.
- MongoDB/Mongoose used as the persistence boundary for application entities.
- Dedicated FAQ and authentication route groups.
- A separate AI service path for generating FAQ content.
- API testing screens showing authentication, FAQ operations, and AI-related requests.

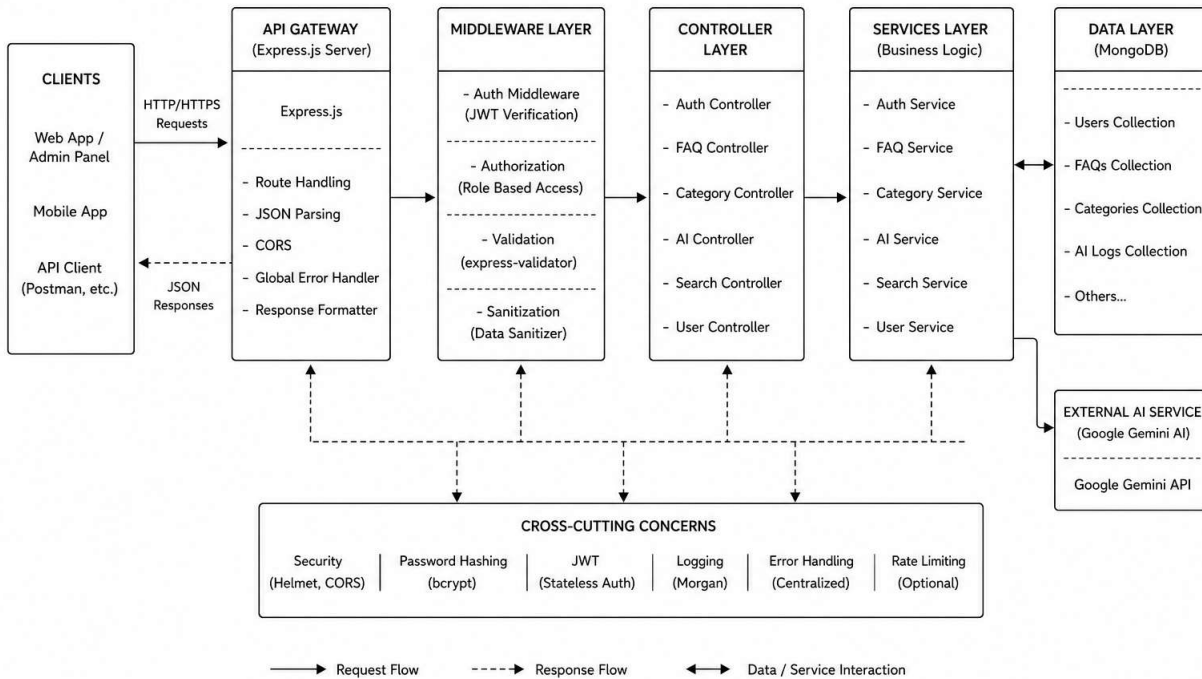


Project source structure visible in the supplied editor screenshot.

2. System Architecture

The architecture diagram shows a request moving from a client-facing API gateway through middleware and controllers into business services and the database layer. Cross-cutting concerns sit beneath the main flow, covering security, validation, error handling, logging and related infrastructure behavior.

AI FAQ Assistant API – Technical Architecture



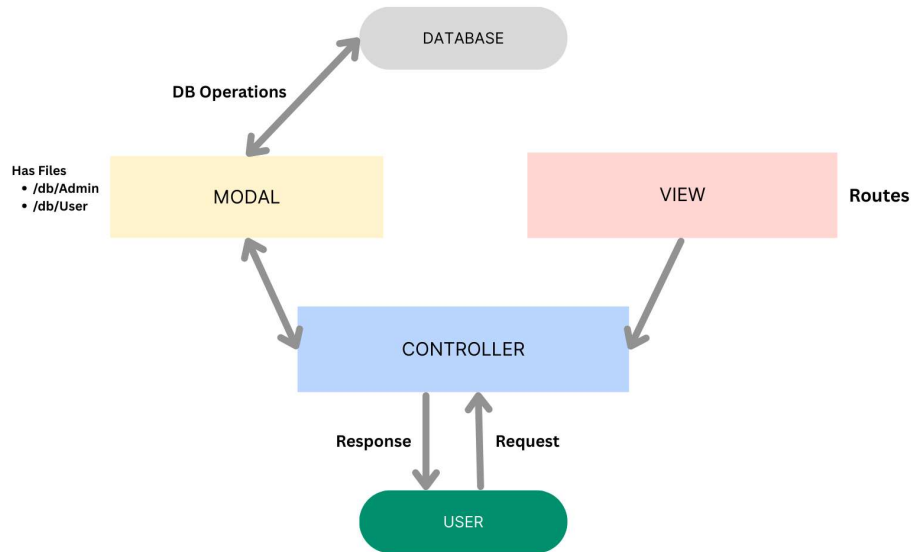
High-level technical architecture.

Layer responsibilities

Layer	Observed responsibility	Why it matters
API Gateway	Receives HTTP requests and routes them into the application.	Creates a consistent entry point.
Middleware	Authentication and request-level controls.	Keeps security checks reusable.
Controller	Coordinates a request and selects the required service/model work.	Separates transport logic from domain behavior.
Service	Encapsulates operations such as FAQ or AI processing.	Makes complex operations easier to maintain.
Data Layer	Persists and retrieves MongoDB documents.	Provides durable application state.

3. MVC Request Model

The supplied MVC diagram depicts the application as a loop between user requests, routing, controllers, models, and the database. A controller receives the request, works with the model or database-facing layer, and returns a response to the caller.



MVC relationship represented in the supplied diagram.

Practical interpretation

Routes: Define which HTTP path and method should handle a request.

Controllers: Coordinate validation, authorization context, service calls and response creation.

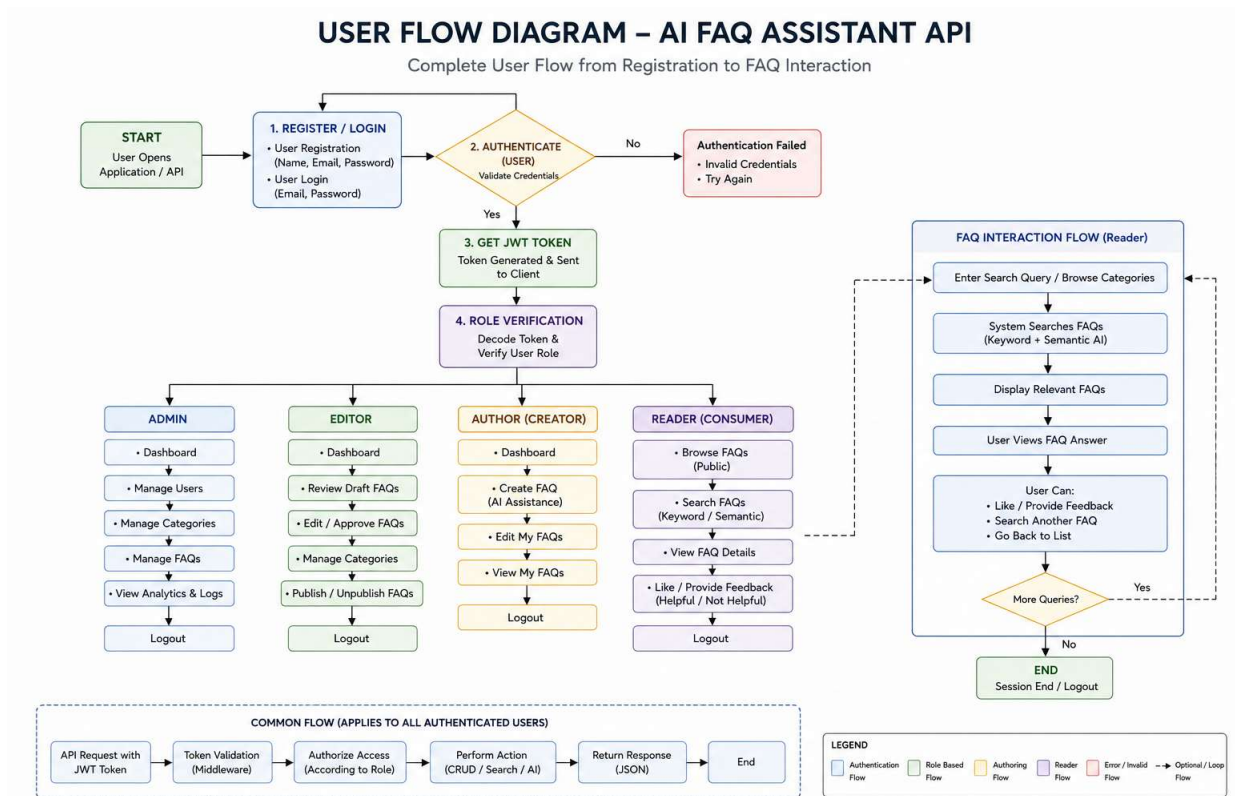
Models: Represent the structure and validation rules for stored documents.

Database: Stores application records and supplies them back to the model layer.

User: Acts as the external initiator of the request/response cycle.

4. User and API Workflow

The supplied user-flow image maps the system from application entry through authentication and protected operations. It also separates public behavior from actions that require an authenticated session.



User flow for the AI FAQ Assistant.

Workflow narrative

01 Entry — A client reaches the backend through an API request.

02 Authentication — Registration or login establishes the user's identity and session context.

03 Authorization — Protected operations pass through the authentication guard before business logic runs.

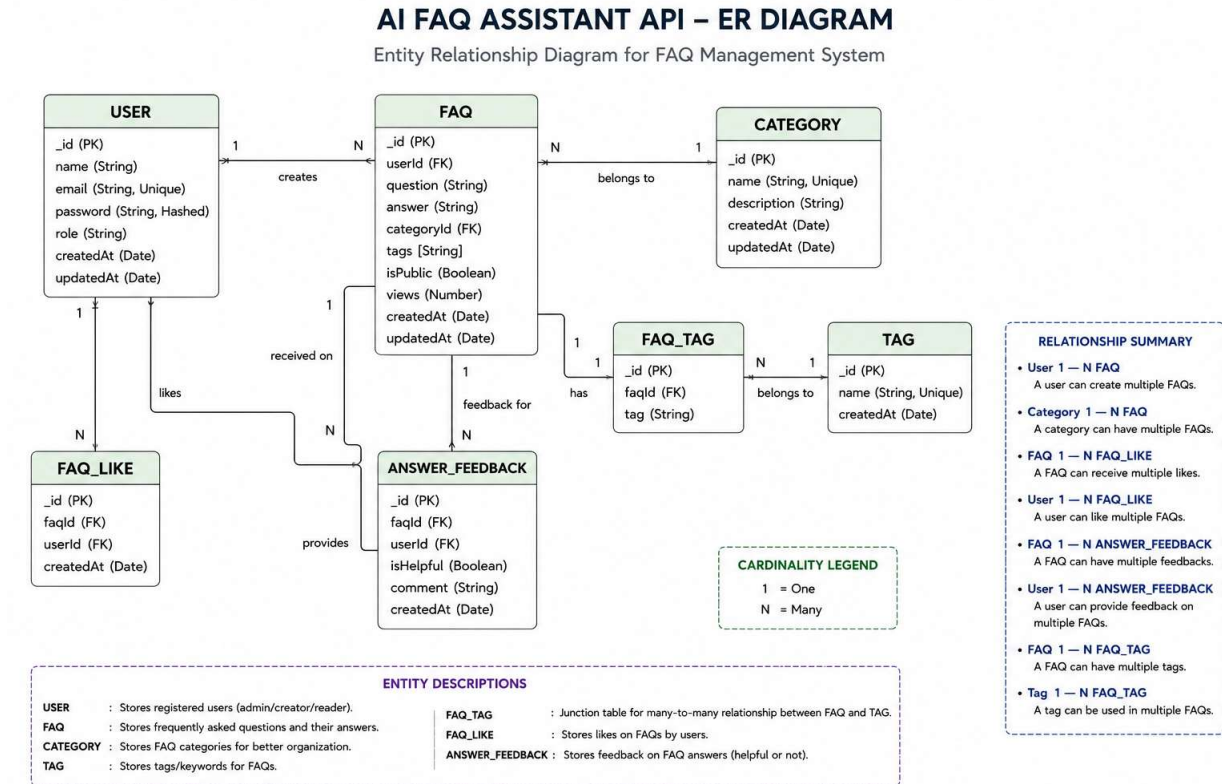
04 FAQ work — The user can create, update, retrieve, delete, or search FAQ information depending on the route.

05 AI assistance — An authenticated request can invoke the AI generation path for FAQ content.

06 Response — The backend returns structured data or an error response to the client.

5. Data Model and Relationships

The ER diagram identifies the principal data objects used by the application: users, FAQs, categories, tags, FAQ-like generated content and supporting logical relationships. The diagram also shows ownership and categorization links around the FAQ record.



Entity-relationship diagram from the supplied project images.

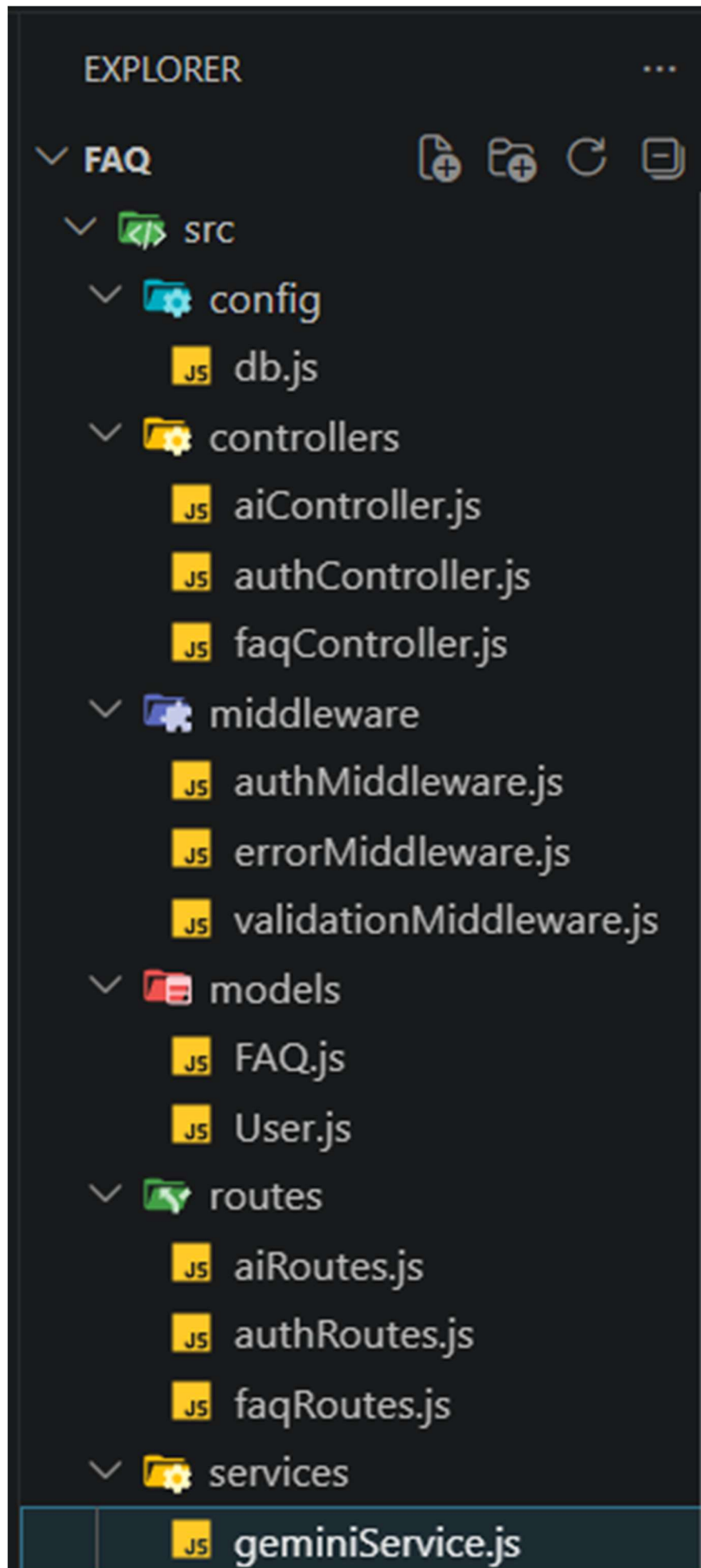
Core entities

- **User** — represents an authenticated account and its profile information.
- **FAQ** — stores the question/answer content together with category and ownership information.
- **Category** — groups FAQ records into meaningful content areas.
- **Tag** — provides an additional way to classify or discover FAQ content.
- **AI-generated content** — represents material produced before it is used as application content.

The visible relationship pattern suggests that ownership and classification are central to the data design: a user can be associated with FAQ records, while categories and tags provide organization and discovery.

6. Backend Source Organization

The project tree shown in the editor is divided into conventional backend modules. The visible folders include controllers, middleware, models, routes, services, configuration and supporting application files.



Visible backend folder structure.

Module map

controllers — Coordinates application operations and prepares API responses.

middleware — Holds reusable request-processing and security logic.

models — Defines database-facing document structures and validation.

routes — Connects HTTP endpoints to controller functions.

services — Contains reusable business or integration operations, including AI-related work.

config / utilities — Supports environment and application setup.

7. Authentication and Route Protection

The authentication middleware screenshot shows a bearer-token flow. The middleware reads the authorization header, extracts the token, verifies it, and uses the decoded identity to continue or reject the request.

```
> middleware > js authMiddleware.js > ...
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const protect = async (req, res, next) => {
5    let token;
6
7    if (
8      req.headers.authorization &&
9      req.headers.authorization.startsWith('Bearer')
10   ) {
11     try {
12       // Get token from header
13       token = req.headers.authorization.split(' ')[1];
14
15       // Verify token
16       const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_long_and_se
17
18       // Get user from the token, exclude password
19       req.user = await User.findById(decoded.id).select('-password');
20
21       if (!req.user) {
22         return res.status(401).json({
23           success: false,
24           message: 'Not authorized, user not found',
25         });
26       }
27
28       next();
29     } catch (error) {
30       console.error(error);
31       return res.status(401).json({
32         success: false,
33         message: 'Not authorized, token failed',
34       });
35     }
36   }
37 }
```

Authentication middleware implementation.

Authentication sequence

1. Read the Authorization header from the incoming request.
2. Confirm that a bearer token is present.
3. Verify the token using the configured signing secret.
4. Resolve the user associated with the token.
5. Attach user context to the request for downstream handlers.
6. Return an authorization error when the token is missing, invalid, or unusable.

This design keeps authentication checks in one reusable middleware component rather than duplicating token logic inside every controller.

8. API Route Design

The route screenshots show separate endpoint groups for authentication, FAQ operations, and AI operations. The route modules import controller functions and attach middleware where a request requires protection.

```

src > routes > authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22

```

Authentication route definitions.

```

src > routes > faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25

```

FAQ route definitions.

```
rc > routes > aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFaq,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10   validateAIFaq,
11 } = require('../middleware/validationMiddleware');
12
13 // Protected routes to prevent API overuse/abuse
14 router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 router.post('/generate-faq', protect, validateAIFaq, generateAIFaq);
16
17 module.exports = router;
18
```

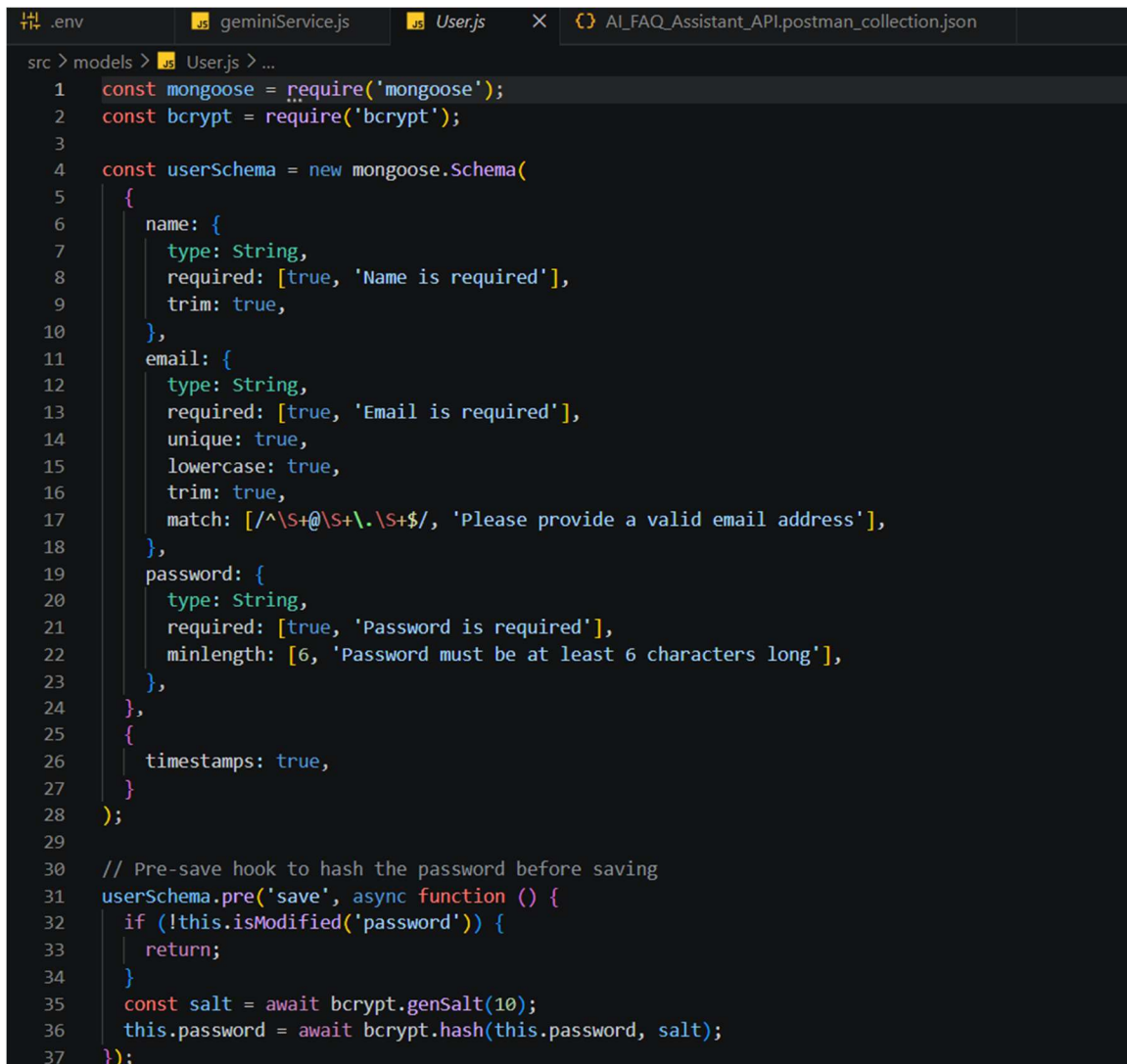
AI route definitions.

Route grouping

Route group	Typical purpose	Access pattern
Authentication	Registration, login and profile-related access.	Public entry plus protected profile operations.
FAQ	Create and manage FAQ resources and retrieve/search them.	Mixed public and protected operations.
AI	Generate FAQ or answer content with the AI service.	Protected operation.

9. Database Models and Validation

The model screenshots show Mongoose schemas with required fields, type constraints, defaults and validation messages. The user model also contains password-processing logic before persistence.

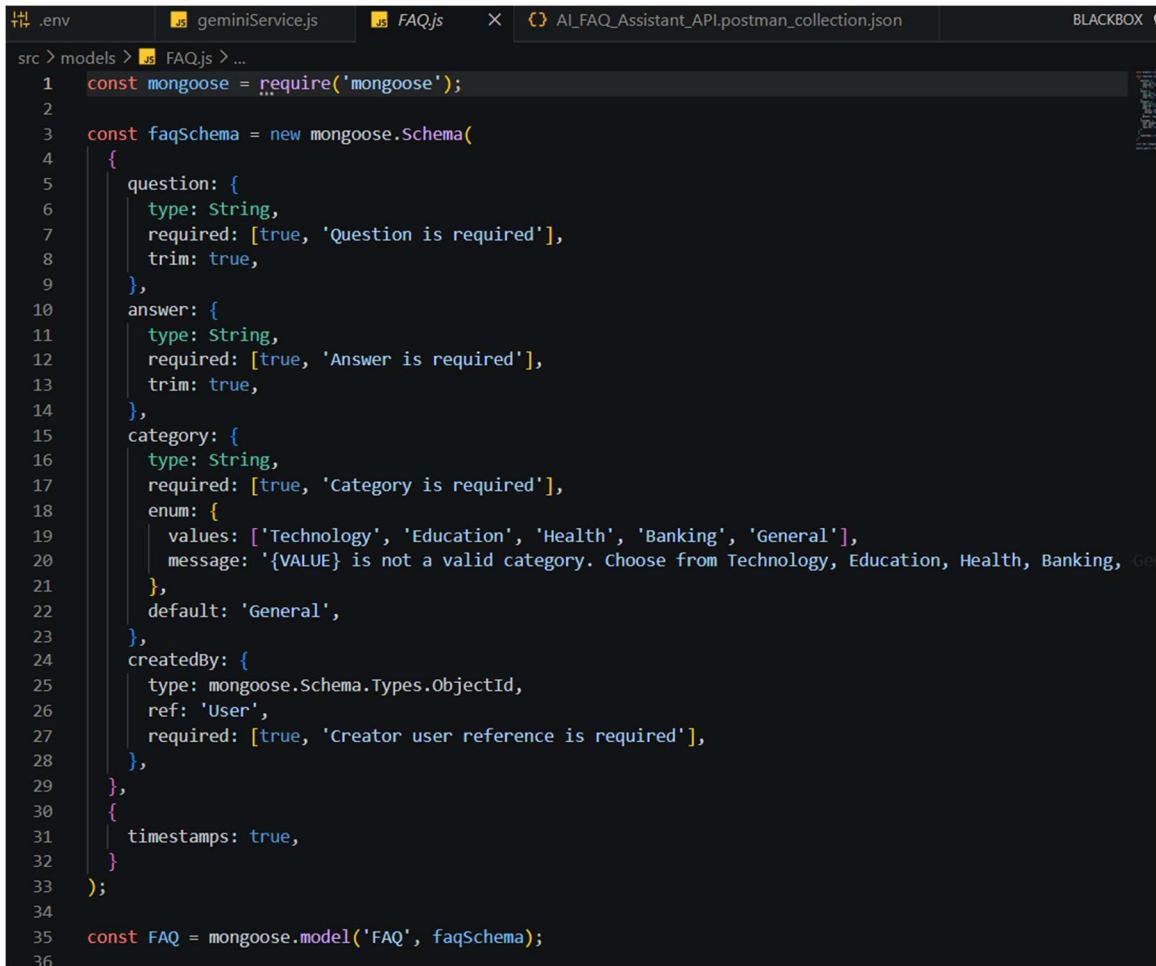


```

src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5    {
6      name: {
7        type: String,
8        required: [true, 'Name is required'],
9        trim: true,
10     },
11     email: {
12       type: String,
13       required: [true, 'Email is required'],
14       unique: true,
15       lowercase: true,
16       trim: true,
17       match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
18     },
19     password: {
20       type: String,
21       required: [true, 'Password is required'],
22       minlength: [6, 'Password must be at least 6 characters long'],
23     },
24   },
25   {
26     timestamps: true,
27   }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });

```

User model and validation logic.



```

src > models > .env  geminiService.js  FAQ.js  X  AI_FAQ_Assistant_API.postman_collection.json  BLACKBOX
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking, Gen
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36

```

FAQ model and validation logic.

Observed validation approach

- Required properties prevent incomplete records from being stored.
- String fields use type and length constraints where appropriate.
- Email input is validated before persistence.
- FAQ fields require the main question, answer and classification information.
- Schema hooks support secure handling of sensitive account data.

10. Application Startup and Error Handling

The server screenshot shows environment/configuration loading, database connection startup, an HTTP port, and a centralized application start function. A separate middleware image demonstrates standardized error processing for API failures.

```
c > server.js > ...
1  const app = require('./app');
2  const connectDB = async () => {
3    try {
4      const dbConnect = require('./config/db');
5      await dbConnect();
6    } catch (error) {
7      console.error('Failed to connect to the database', error);
8    }
9  };
10
11 const startServer = async () => {
12   // Connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error(`Unhandled Rejection: ${err.message}`);
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30
```

Application server startup.

```

> middleware > .js errorMiddleware.js > ...
1  const errorHandler = (err, req, res, next) => {
2    let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3    let message = err.message;
4
5    // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6    if (err.status) {
7      statusCode = err.status;
8    }
9
10   // Mongoose duplicate key error (e.g. duplicate email)
11   if (err.code === 11000) {
12     statusCode = 400;
13     const field = Object.keys(err.keyValue)[0];
14     message = `Duplicate field value entered: ${field}. Please use another value.`;
15   }
16
17   // Mongoose validation error
18   if (err.name === 'ValidationError') {
19     statusCode = 400;
20     message = Object.values(err.errors)
21       .map((val) => val.message)
22       .join(', ');
23   }
24
25   // Mongoose bad ObjectId
26   if (err.name === 'CastError') {
27     statusCode = 404;
28     message = `Resource not found with id of ${err.value}`;
29   }
30
31   // Only log stack trace to the console for unexpected 500 Server Errors
32   if (statusCode === 500) {
33     console.error(err.stack);
34   } else {

```

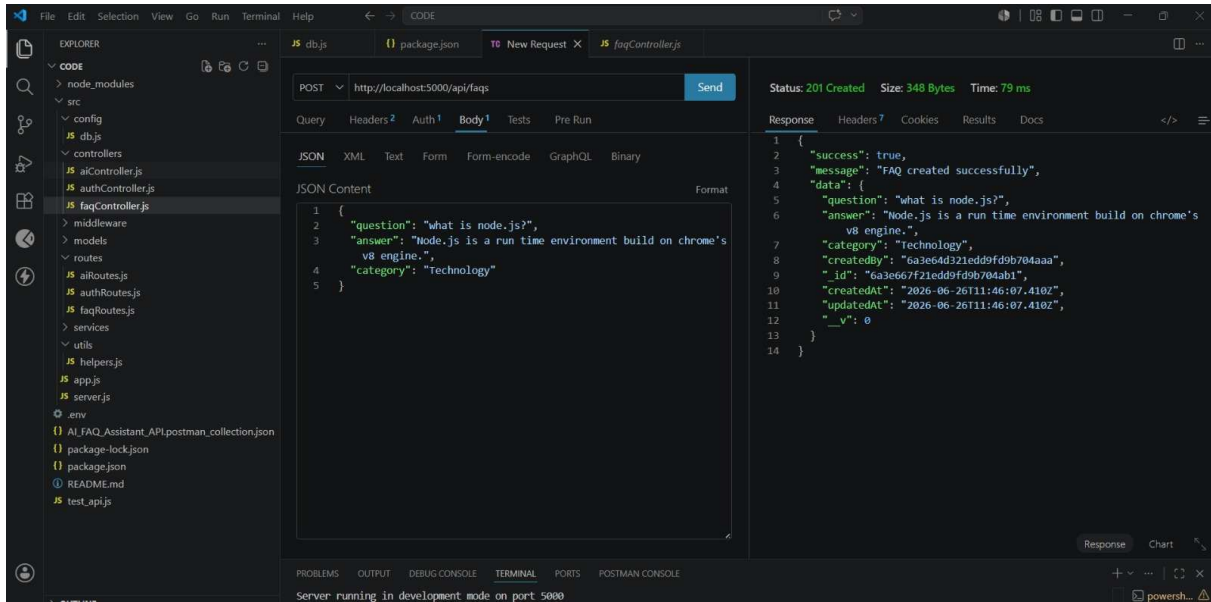
Centralized error-handling middleware.

Operational pattern

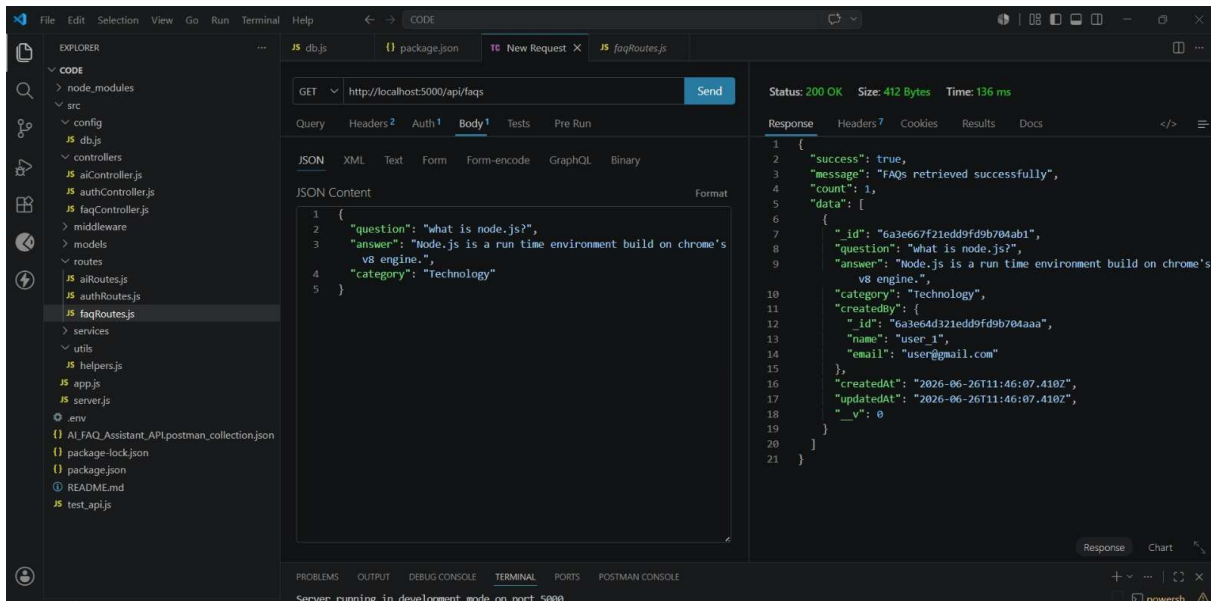
The application initializes its dependencies, connects to the database, starts the Express server, and uses centralized error handling so that failures can be converted into consistent API responses.

11. API Testing Evidence

The supplied API-client screenshots provide visual evidence of endpoint testing. They show request methods, JSON payloads and response areas, indicating that the backend is being exercised through a REST client rather than only through direct browser access.



Authentication-related API test.



FAQ API test.

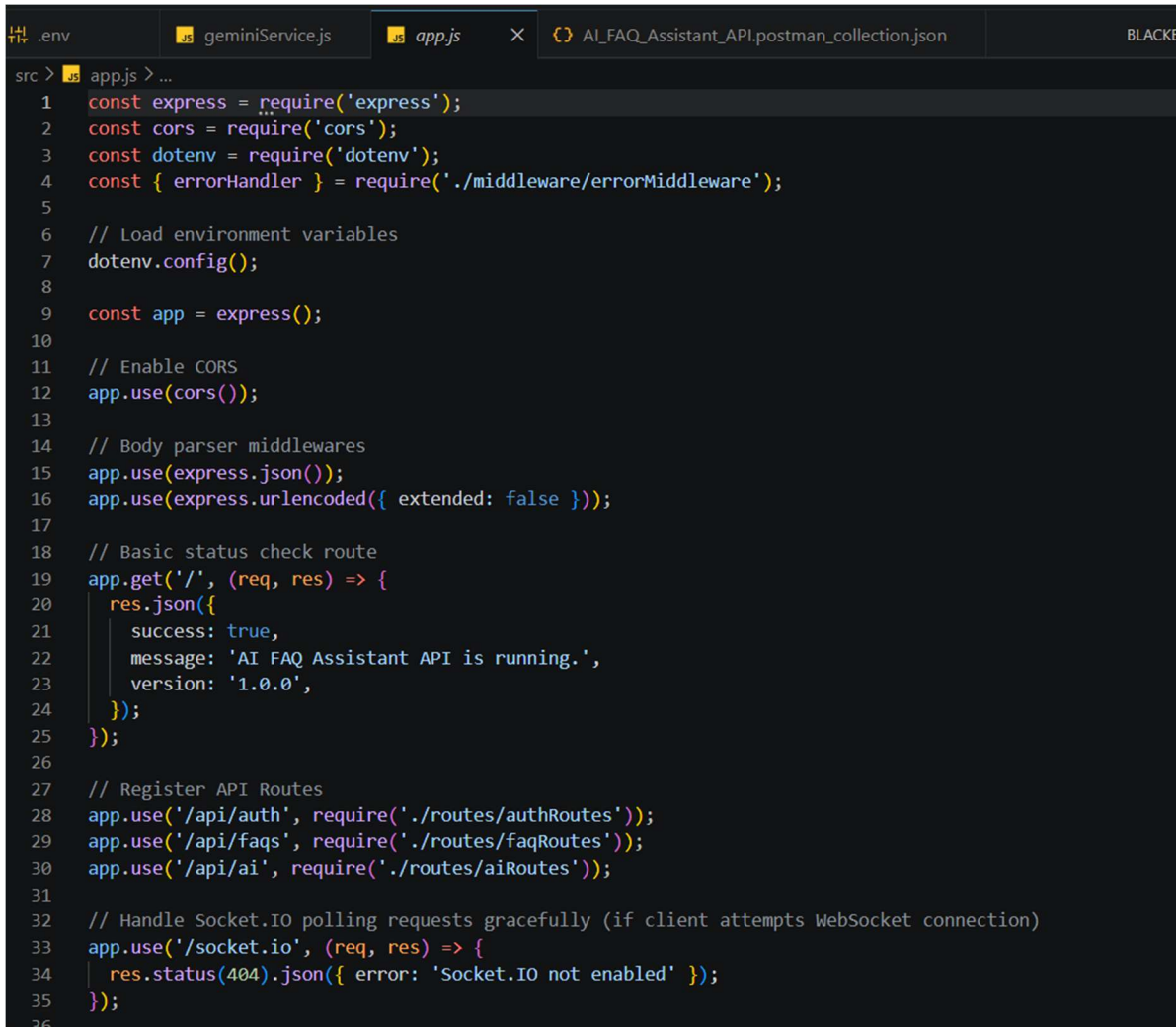
Testing objectives

- Confirm that requests reach the intended endpoint.
- Check that request bodies match the expected schema.
- Verify authentication behavior for protected routes.
- Inspect returned status codes and JSON responses.

- Validate that database-backed operations produce the expected records.

12. AI-Assisted FAQ Generation

The AI service screenshot shows a dedicated integration layer that prepares a prompt/request and communicates with an external generative AI SDK. The route layer exposes this capability through a protected API operation.



```

src > .env  geminiService.js  app.js  AI_FAQ_Assistant_API.postman_collection.json  BLACKB
src > .js app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const dotenv = require('dotenv');
4  const { errorHandler } = require('./middleware/errorMiddleware');
5
6  // Load environment variables
7  dotenv.config();
8
9  const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36

```

AI service integration shown in the source code.

API test for AI-assisted content generation.

Conceptual AI flow

7. An authenticated client submits a topic or content-generation request.
8. The AI route forwards the request to the relevant controller/service.
9. The AI service builds the generation request and calls the configured model SDK.
10. The generated response is returned to the application layer.
11. The result can then be presented to the client or used as FAQ content.

13. Security and Reliability Design

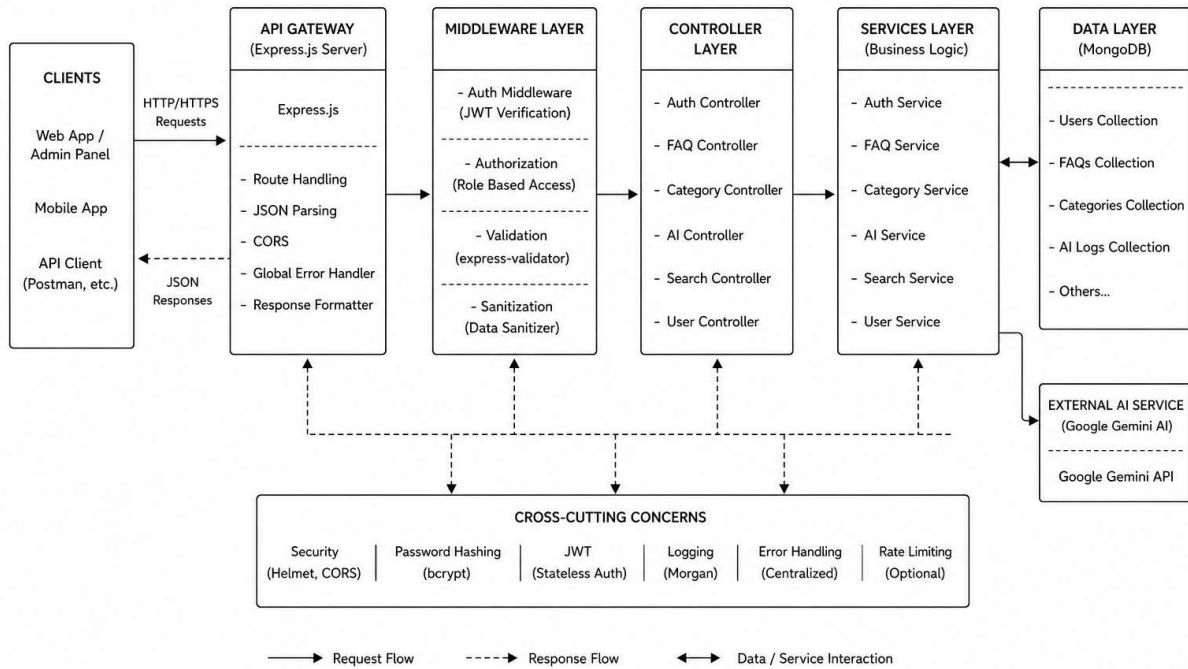
The supplied visuals collectively show several controls intended to make the API safer and more predictable. These controls are distributed across middleware, models, controllers and centralized error processing.

Control	Role
Token protection	Private routes are guarded through bearer-token authentication.
Password handling	The user model includes password-processing logic before data is saved.
Schema validation	Mongoose schemas reject structurally invalid or incomplete data.
Centralized errors	A dedicated error middleware standardizes failure handling.
Route separation	Public and protected operations are visibly separated in route definitions.
Layer isolation	Authentication, business logic, persistence and AI integration are kept in distinct modules.

14. End-to-End Operation

Taken together, the supplied images describe a backend in which a client request travels through a predictable series of boundaries. Authentication determines whether a protected operation may proceed; controllers and services perform the requested work; models validate database records; and the API returns the resulting data.

AI FAQ Assistant API – Technical Architecture



Architecture view of the end-to-end system.

Example: creating an FAQ

12. Client sends a protected POST request containing FAQ fields.
13. Authentication middleware verifies the bearer token.
14. The route delegates the request to the FAQ controller.
15. The controller validates the input through the FAQ model/service path.
16. MongoDB receives the validated document.
17. The API returns the created FAQ or a standardized error.

Example: generating an AI FAQ

18. Client sends a protected generation request with a topic.
19. The AI route applies the authentication guard.
20. The controller delegates to the AI service.
21. The service communicates with the configured generative model.
22. The generated content is returned to the API client for further use.

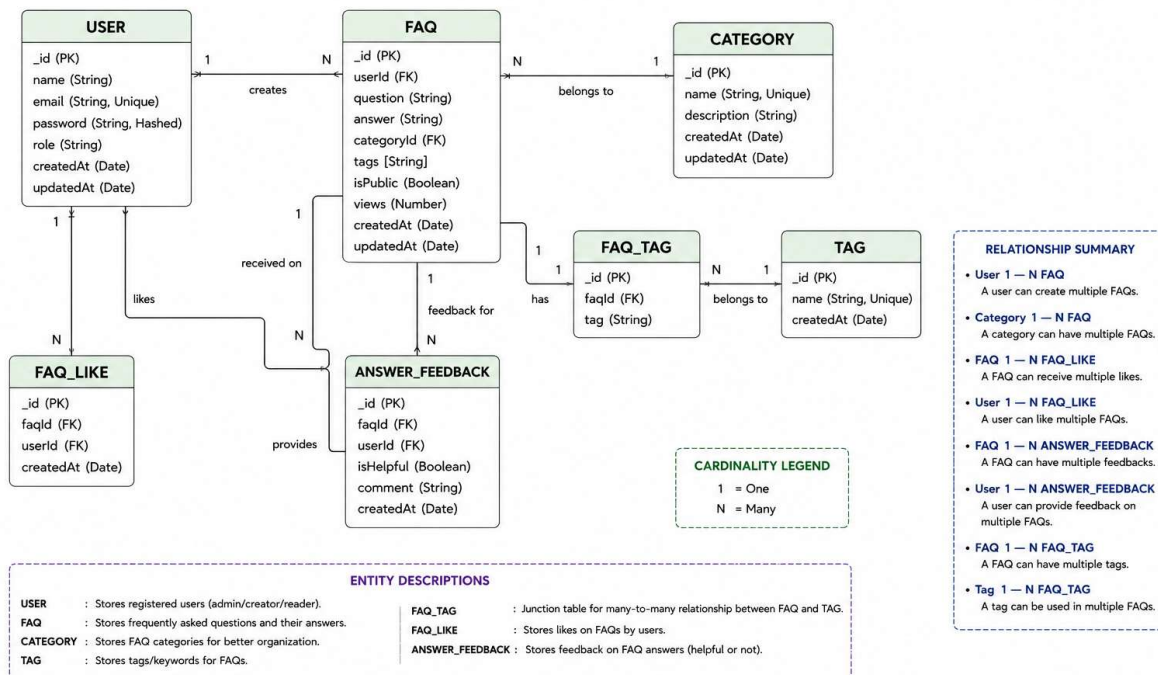
15. Conclusion

The supplied images present a modular AI FAQ backend with clear separation between transport, security, business logic, data modeling and AI integration. The diagrams explain the overall architecture and data relationships, while the editor and API-client screenshots demonstrate how those concepts are implemented and tested.

As an image-derived project report, this document intentionally uses newly written explanatory content rather than reproducing the original document text. The descriptions are limited to structures and behaviors that can be observed from the supplied visuals.

AI FAQ ASSISTANT API – ER DIAGRAM

Entity Relationship Diagram for FAQ Management System



Final reference: project data model.

END OF REPORT